🔒 CONFIDENCIAL

# Reporte de Vulnerabilidad

## Inyección SQL de segundo orden a través de metadatos EXIF de imágenes (campo Make)

| | |
|---|---|
| ID del Ticket | **VULN-2026-010** |
| Activo evaluado | **Aplicación web «Galería de Imágenes» (Lab Galería)** |
| Severidad | **ALTA** **CVSS v3.1 8.1** |
| Tipo | **SQL Injection de segundo orden (vía EXIF) — CWE-89 / CWE-20** |
| Cliente | **Software Seguro — Laboratorio Galería de Imágenes** |
| Analista | **J. M. F. · jmf01111974@gmail.com** |
| Fecha del reporte | **08 de noviembre de 2026** |
| Versión | **1.0** |

## Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

# 1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

### ¿Qué encontramos?

La galería muestra, junto a cada foto, datos tomados de la propia imagen (como el **fabricante de la cámara**). El problema es que la aplicación **confía en esa información incrustada en el archivo** y la usa directamente para armar sus consultas a la base de datos. Como ese dato lo controla quien sube la foto, se puede **esconder dentro de la imagen un comando malicioso** que la aplicación termina ejecutando. Con esa técnica se logró extraer información interna de la base de datos.

### ¿Por qué importa para el negocio?

Es una **inyección SQL**, una de las fallas más graves de una aplicación web, con el agravante de que el ataque entra por un canal inesperado: los **metadatos de un archivo subido**. Habitualmente permite **leer, modificar o borrar cualquier dato** de la base (usuarios, credenciales, contenido). Para un sitio que recibe archivos de usuarios, esto implica un riesgo de compromiso de datos y de la plataforma, con las consiguientes consecuencias legales y reputacionales.

### ¿Qué tan grave es?

**Severidad ALTA — CVSS 8.1.** La explotación es remota y sencilla con herramientas estándar (`exiftool` + Burp). Un carácter ("comilla") colocado en los metadatos provocó una respuesta anómala de la base de datos, confirmando la falla; a partir de ahí se extrajo el **nombre del archivo de base de datos** SQLite, obteniéndose el código de verificación del laboratorio.

### ¿Qué hay que hacer?

Tratar los metadatos de los archivos subidos como **entrada no confiable** y usar **consultas parametrizadas** en todo el código, de modo que ningún dato del usuario (ni siquiera el proveniente de un archivo) se mezcle con las instrucciones SQL.

| Riesgo | Probabilidad | Impacto | Urgencia de acción |
|---|---|---|---|
| Lectura/alteración de la base de datos mediante inyección SQL incrustada en metadatos de imágenes | Alta | Alto | **Inmediata** |

# 2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

### 2.1 Descripción de la vulnerabilidad

Al procesar una imagen subida, la aplicación extrae metadatos EXIF —en particular el campo `Make` (fabricante)— y los incorpora a una consulta SQL (SQLite) **por concatenación de cadenas**, sin parametrización ni validación. Como el valor de `Make` es controlable por quien sube la foto (editable con `exiftool`), se habilita una **SQL Injection de segundo orden** (CWE-89): el dato malicioso se almacena primero y la inyección se materializa cuando la aplicación lo usa para construir la consulta.

La consulta subyacente maneja **dos valores** (el identificador de la imagen y el texto del fabricante), lo que permitió ajustar un `UNION SELECT` con el número de columnas correcto para exfiltrar datos arbitrarios.

### 2.2 Flujo del ataque

- El atacante observa que la galería muestra el campo `Make` (p. ej. "NIKON CORPORATION") tomado del EXIF.
- Con `exiftool` modifica el campo `Make` de una imagen propia e inserta una comilla simple; al subirla, la base de datos responde de forma anómala (inyección confirmada).
- Determina que la consulta espera dos columnas (id + texto) y arma un `UNION SELECT` acorde.
- Inyecta una subconsulta SQLite que devuelve el nombre del archivo de base de datos y lo visualiza en la galería, donde debería aparecer el fabricante.

### 2.3 Clasificación

| Categoría | Referencia |
|---|---|
| **Tipo (CWE)** | CWE-89 — SQL Injection · CWE-20 — Improper Input Validation (metadatos de archivo no confiables) · CWE-209 — Generation of Error Message Containing Sensitive Information |
| **Subtipo** | Second-Order (Stored) SQL Injection — la entrada se almacena y la inyección se ejecuta posteriormente |
| **OWASP Top 10 (2021)** | A03:2021 — Injection |
| **OWASP API Security Top 10 (2023)** | API8:2023 — Security Misconfiguration |
| **CVE** | No aplica (defecto propio de la aplicación, no de un componente de terceros) |

### 2.4 Activos y alcance afectado

| Elemento | Detalle |
|---|---|
| **Aplicación** | «Galería de Imágenes» (Lab Galería) — base de datos SQLite |
| **Host** | `chl-<instancia>-galeria.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión) |
| **Vector de entrada** | Metadato EXIF `Make` de la imagen subida (procesado e insertado en la consulta) |
| **Parámetro** | Campo `Make` (fabricante), controlable con `exiftool` |
| **Dato sensible involucrado** | Nombre del archivo de base de datos SQLite y, potencialmente, todo su contenido |
| **Alcance** | Lectura (y potencial escritura) de datos de la base según los privilegios de la aplicación |

### 2.5 Causa raíz

La consulta SQL se construye por **concatenación de cadenas** con un dato controlable por el usuario, y la aplicación **asume que los metadatos de un archivo son confiables**. La validación de entrada no se extiende a los campos EXIF, por lo que un valor diseñado como payload altera la sintaxis y la lógica de la consulta. El uso de SQLite y la función `pragma_database_list` permiten, además, revelar información interna (el archivo de la base).

# 3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

| CVSS Base Score | Severidad | Vector |
|---|---|---|
| **8.1** | ALTA | `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N` |

### Desglose del vector (métricas base)

| Métrica | Valor | Justificación |
|---|---|---|
| Attack Vector (AV) | **Network (N)** | La aplicación se explota de forma remota a través de Internet (HTTP/HTTPS). |
| Attack Complexity (AC) | **Low (L)** | No requiere condiciones especiales: editar un metadato con exiftool y subir la imagen. |
| Privileges Required (PR) | **Low (L)** | Se requiere la capacidad de subir imágenes (cuenta de fotógrafo/usuario). |
| User Interaction (UI) | **None (N)** | El atacante sube su propia imagen y visualiza el resultado; no depende de una víctima. |
| Scope (S) | **Unchanged (U)** | El impacto permanece dentro de la misma aplicación/base de datos. |
| Confidentiality (C) | **High (H)** | Permite leer información interna de la base (nombre del archivo) y, potencialmente, todo su contenido. |
| Integrity (I) | **High (H)** | La inyección otorga control sobre la consulta, habilitando modificación de datos según los privilegios de la aplicación. |
| Availability (A) | **None (N)** | No se demostró impacto en la disponibilidad (ver nota). |

Nota de criterio: se asigna `A:N` por no haberse demostrado impacto en disponibilidad. Si la aplicación permitiera operaciones destructivas sobre el archivo SQLite (`A:H`), el vector `AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` alcanza **8.8**; y si la carga de imágenes fuera anónima (`PR:N`), llegaría a **9.8 (Crítica)**.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

# 4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con una cuenta del laboratorio, exiftool y Burp puede replicar el hallazgo.

### 4.1 Requisitos previos

- Capacidad de subir imágenes en la galería del laboratorio.
- `exiftool` para ver y modificar metadatos EXIF; Burp Suite o navegador.
- Una imagen de prueba (JPEG) con campo `Make`.

## 4.2 Pasos de reproducción

**Paso 1 — Inspeccionar los metadatos.** Ver el valor del campo `Make` que la galería muestra.

```
exiftool -Make foto.jpg
# -> Make : NIKON CORPORATION
```

**Paso 2 — Confirmar la inyección (comilla en el metadato).** Modificar `Make` con una comilla simple y subir la imagen; la base de datos responde de forma anómala.

```
exiftool -Make="Nikon'" foto.jpg
# Subir foto.jpg a la galería -> respuesta anómala / error SQL -> inyección presente
```

*Evidencia 1 — La comilla en el campo `Make` altera la consulta: SQL Injection de segundo orden confirmada.*

**Paso 3 — Ajustar el número de columnas.** Se determina que la consulta maneja dos valores (id de imagen + texto), por lo que el `UNION SELECT` debe devolver dos columnas.

**Paso 4 — Extraer el nombre del archivo de base de datos (SQLite).** Inyectar una subconsulta a `pragma_database_list` en el campo `Make`.

```
exiftool -Make="x' UNION SELECT 1,(SELECT file FROM pragma_database_list WHERE name=\"main\")-- -"
foto.jpg

# Subir la imagen y abrir su detalle en la galería:
# donde debería verse el fabricante, se muestra la ruta/nombre del archivo SQLite
```

*Evidencia 2 — El valor de `Make` en la galería muestra el nombre del archivo de base de datos, confirmando la exfiltración vía SQLi.*

**Paso 5 — Verificación del impacto.** Se obtiene el nombre especial del archivo y el laboratorio entrega el código.

```
# Código de verificación obtenido:
#   3f8ca106a5118b3c418ec00907120d6a
```

*Evidencia 3 — Objetivo cumplido: se extrajo información interna de la base de datos mediante un metadato de imagen.*

> Nota técnica: confirmada la inyección, un atacante podría iterar con `sqlite_master` (esquema y tablas) y `UNION` para extraer otras tablas (usuarios, credenciales). En esta PoC la demostración se limitó a leer el nombre del archivo de base de datos; no se realizó extracción masiva.

## 4.3 Resultado esperado vs. obtenido

| Esperado (sistema seguro) | Obtenido (sistema vulnerable) |
|---|---|
| Los metadatos se tratan como datos (consulta parametrizada); una comilla o un `UNION` en `Make` no alteran la consulta. | El campo `Make` inyecta SQL y permite exfiltrar el nombre del archivo de base de datos SQLite. |

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado, con el impacto mínimo necesario para demostrar el riesgo. No se extrajeron datos masivamente ni se alteraron registros.

# 5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

## 5.1 Solución recomendada (prioritaria)

**Consultas parametrizadas, también para los metadatos.** Cualquier valor extraído de un archivo (EXIF incluido) debe pasar como parámetro vinculado, nunca concatenarse en el SQL.

```
# VULNERABLE: concatena el metadato Make en la consulta
make = exif.get("Make", "desconocido")
cursor.execute("INSERT INTO fotos (img_id, fabricante) VALUES (" + img_id + ", '" + make + "')")

# CORREGIDO: parámetros vinculados (la BD trata make como dato)
make = exif.get("Make", "desconocido")
cursor.execute(
    "INSERT INTO fotos (img_id, fabricante) VALUES (?, ?)",
    (img_id, make),
)
```

## 5.2 Mitigaciones y buenas prácticas

- **Tratar los metadatos como entrada no confiable**: validar longitud y formato del campo `Make` (allow-list de caracteres) antes de almacenarlo.
- **Sanitizar/normalizar EXIF al subir**: extraer solo los campos necesarios y, si es posible, eliminar el resto de metadatos del archivo servido.
- **Deshabilitar errores de BD verbosos** en producción (CWE-209); registrar el detalle solo del lado del servidor.
- **Mínimo privilegio**: el proceso de la aplicación con los permisos imprescindibles sobre el archivo SQLite; considerar solo-lectura donde aplique.
- **Revisión de código / SAST** en todos los puntos donde se procesan archivos subidos y sus metadatos.

## 5.3 Plan de acción

| Acción | Responsable | Prioridad | Plazo | Estado |
|---|---|---|---|---|
| Parametrizar todas las consultas que usan metadatos EXIF | Equipo de desarrollo | Crítica | Inmediato | Pendiente |
| Validar/sanitizar metadatos al subir (allow-list) y stripping de EXIF | Equipo de desarrollo | Alta | 7–15 días | Pendiente |
| Deshabilitar errores verbosos y aplicar mínimo privilegio en SQLite | Desarrollo / SecOps | Alta | 24–72 h | Pendiente |
| Revisar el resto del código por SQLi (incl. canales de segundo orden) | Desarrollo / SecOps | Alta | 15 días | Pendiente |
| Test de regresión (payloads en EXIF = sin efecto) y re-test | QA / Seguridad | Alta | Con el fix | Pendiente |

### 5.4 Verificación (re-test)

Volver a subir imágenes con payloads en el campo `Make` (comilla, `UNION SELECT`, `pragma_database_list`). El criterio de aceptación es que el valor se almacene y muestre como texto literal, sin alterar la consulta ni exponer errores ni datos internos de la base. Confirmar que la funcionalidad legítima (mostrar el fabricante real o "desconocido") sigue operando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que la inyección vía metadatos ya no es posible, sin regresiones en la funcionalidad legítima.

## 6. Referencias e Historial

### Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- CWE-89 — SQL Injection: https://cwe.mitre.org/data/definitions/89.html
- CWE-20 — Improper Input Validation: https://cwe.mitre.org/data/definitions/20.html
- CWE-209 — Generation of Error Message Containing Sensitive Information: https://cwe.mitre.org/data/definitions/209.html
- OWASP — SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy — SQL injection (second-order): https://portswigger.net/web-security/sql-injection
- SQLite — PRAGMA database_list / pragma functions: https://www.sqlite.org/pragma.html#pragma_database_list
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

### Historial de cambios

| Versión | Fecha | Autor | Cambio |
|---|---|---|---|
| 1.0 | 2026-11-08 | J. M. F. | Creación del reporte: SQL Injection de segundo orden vía metadatos EXIF (campo Make) con exfiltración del nombre de la base SQLite. |

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.